Projektuppgift

Introduktion till webbutveckling med HTML, CSS och JavaScript, DT224G - HT2026

Micael Huczkowski



Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Micael Huczkowski, mihu2501@student.miun.se
Utbildningsprogram: TWEUG, Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

1 Sammanfattning

Projektet Kalender Gratis har genomförts med målet att skapa en avskalad och responsiv svensk kalendertjänst utan annonser och onödiga distraktioner. Arbetet inleddes med en jämförelse av befintliga kalendertjänster och planering i form av wireframes och en designskiss för mobil och desktop. Webbplatsen utvecklades därefter stegvis med HTML, CSS och JavaScript samt versionshanterades med Git. Resultatet är en webbplats bestående av en månadsvy, en årsvy och en sida med information och kontaktformulär. Kalendern beräknar datum och veckonummer dynamiskt och hämtar information om svenska helgdagar, högtidsaftnar och namnsdagar från ett externt API. Användaren kan navigera mellan månader och år samt växla mellan månads- och årsvyn. Webbplatsen innehåller även en mobilmeny, formulärvalidering, tillfällig lagring av redan hämtad kalenderinformation och felhantering om API-tjänsten inte kan nås. Layouten har utvecklats enligt mobile first-principen och anpassas till mobiltelefoner, surfplattor och datorer. HTML och CSS validerades utan fel och tillgänglighetstester med WAVE gav inga rapporterade fel eller kontrastfel. Projektets huvudsakliga mål har därmed uppnåtts och resultatet är en fungerande, responsiv och tillgänglig kalendertjänst med relevant svensk kalenderinformation.

Innehållsförteckning

1 Sammanfattning iii
Innehållsförteckning iv
2 Introduktion 1
2.1 Bakgrund och problemmotivering 1
2.2 Syfte och målgrupp 1
2.3 Avgränsningar 1
2.4 Detaljerad problemformulering 1
3 Teori 2
3.1 Semantisk HTML och tillgänglighet 2
3.2 Responsiv design med CSS 2
3.3 JavaScript, DOM och asynkron data 2
3.4 Versionshantering 2
4 Metod 3
4.1 Förstudie och planering 3
4.2 Iterativ implementation 3
4.3 Verktyg och testmetoder 4
5 Konstruktion 5
5.1 Informationsarkitektur 5
5.2 Grafisk profil och responsiv layout 5
5.3 Beräkning av kalendern 5
5.4 Extern kalenderdata och cache 5
5.5 Status och felhantering 6
5.6 Navigation och formulär 6
5.7 Tillgänglighetsval 6
6 Resultat 7
6.1 Färdig funktionalitet 7
6.1.1 Månadsvy 7
6.1.2 Årsvy 8
6.1.3 Om och Kontakt 10
6.2 Testresultat 12
6.3 Versionshistorik 13
7 Slutsatser 14

2 Introduktion

2.1 Bakgrund och problemmotivering

Det finns ett stort antal kalendertjänster på internet idag som löser samma grundproblem med har olika stilar och upplägg. Vid en sökning på Google så dyker kalender.se som första träff. Den har ett ganska enkelt upplägg på själva kalendern med en god månadsvy, men innehåller väldigt mycket onödiga funktioner och annonser som drar ner upplevelsen. Den andra träffen man får är kalender-365.se som har en bra årsvy men inte lika bra månadsvy, även denna har problem med annonser som stör ut användarupplevelsen.
Projektidén går ut på att kombinera en bra månadsvy med en enkel årsvy för att bygga en enkel kalender som går smidigt och snabbt navigera i och gör det effektivt att kolla upp kollande datum och planera sin tid.

2.2 Syfte och målgrupp

Syftet var att skapa en responsiv svensk kalender för personer som snabbt vill kontrollera datum, veckonummer, helgdagar och namnsdagar. Målgruppen är bred och omfattar användare på mobil, surfplatta och dator. Det kommunikativa målet var att kalendern skulle vara självförklarande och ge en tydlig avsändare utan att belasta användaren med annonser, popupfönster eller kontokrav.

2.3 Avgränsningar

- Ingen inloggning, användarprofiler eller funktioner mot andra kalendrar.
- Det ska inte gå att spara händelser.
- Kontaktformuläret validerar inmatningen men skickar inga uppgifter på riktigt.
- Namnsdagar och röda dagar är beroende av ett externt API och den datan lagras tillfälligt i minnet på användarens webbläsare.

2.4 Detaljerad problemformulering

Projektet ska lösa problemet att många befintliga kalendertjänster upplevs som röriga, reklamtyngda eller svåra att överblicka. Målet är därför att utveckla en sammanhängande svensk kalenderwebbplats med en avskalad och responsiv utformning. Webbplatsen ska bestå av en månadsvy, en årsvy samt en sida med information och kontaktformulär. Månadsvyn ska automatiskt visa aktuell månad och innehålla korrekta datum, veckonummer, svenska helgdagar, högtidsaftnar och namnsdagar. Användaren ska kunna navigera mellan månader och år, och länkarna mellan månadsvyn och årsvyn ska behålla det valda året och den valda månaden. Webbplatsen ska även innehålla en mobilmeny, formulärvalidering och tydlig återkoppling om extern kalenderinformation inte kan hämtas. Lösningen ska fungera på olika skärmstorlekar och följa grundläggande krav på semantik och tillgänglighet.

3 Teori

3.1 Semantisk HTML och tillgänglighet

Semantisk HTML innebär att element innebär att element väljs utifrån innehållets funktion, så som header, nav, main, section, article och footer. HTML standarden beskriver elementen som strukturella delar som gör uppbyggnaden tydligare.[1]

Web Content Accessibility Guidelines (WCAG) 2.2 anger bland anna tatt normal text bör som minst ha kontrastförhållanden 4,5:1 och att tangetbordsfokus ska vara synligt.[2] Dessa principer påverkar färgval, fokusmarkeringar, satusmeddelanden samt den manuella tangenbordstestningen. Automatiska tester ersätter inte manuella tester men ger stöd i att hitta eventuella problem.

3.2 Responsiv design med CSS

Projektet följer mobile-first principen vilket innebär att grundstilarna är anpassade för mindre skärmar och utökas med media queries vid 768 och 1100 pixlar. Media queries blir en central komponent i responsiv design då de möjliggör att styra CSS efter exempelvis vybredd.[3] CSS grid används för att skapa kalenderrutnät och årsöversikt medans Flexbox används för att fördela innehåll längst en axel, exempelvis knappinnehåll och informationsrutor.

3.3 JavaScript, DOM och asynkron data

JavaScript bygger kalenderns datumrutor genom DOM-manipulation. Date-objekt används för att beräkna vilken måndag som ska visas först, när månadsskiftet sker samt veckonummer. Fetch API används för att hämta kalenderresurser över internet och returnerar ett Promise som hanterar asynkront.[4] Lyckade API svar omvandlas från JSON till objekt och arrayer för att sedan matchas mot varje datum. Vid fel så meddelas det om att information inte går att hämta och enbart grundkalender visas.

3.4 Versionshantering

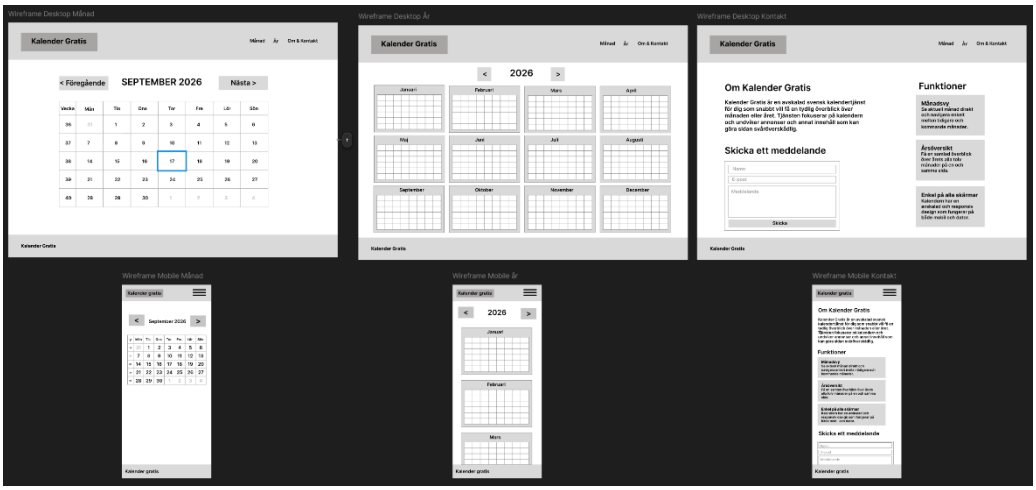
Git är ett distribuerat versionshanteringssystem som sparar förändringar över tid och gör det möjligt att arbeta i separata grenar.[5] Projektet har utvecklats i grenen dev och fördes sedan över till main när projektet var färdigt. Vidare utveckling kommer sedan primärt att ske i dev för att sedan släppas i main vid införande av nya funktioner och förbättringar.

[1] WHATWG. HTML Living Standard: Semantics, structure, and APIs of HTML documents. Tillgänglig: https://html.spec.whatwg.org/multipage/semantics.html [Hämtad 2026-10-29]
[2] World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.2. Tillgänglig: https://www.w3.org/TR/WCAG22/ [Hämtad 2026-10-29].
[3] MDN Web Docs. CSS media queries. Tillgänglig: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries [Hämtad 2026-10-29].
[4] MDN Web Docs. Fetch API. Tillgänglig: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API [Hämtad 2026-10-29].
[5] Chacon S, Straub B. Pro Git, 2nd ed. Git. Tillgänglig: https://git-scm.com/book/en/v2 [Hämtad 2026-10-29].

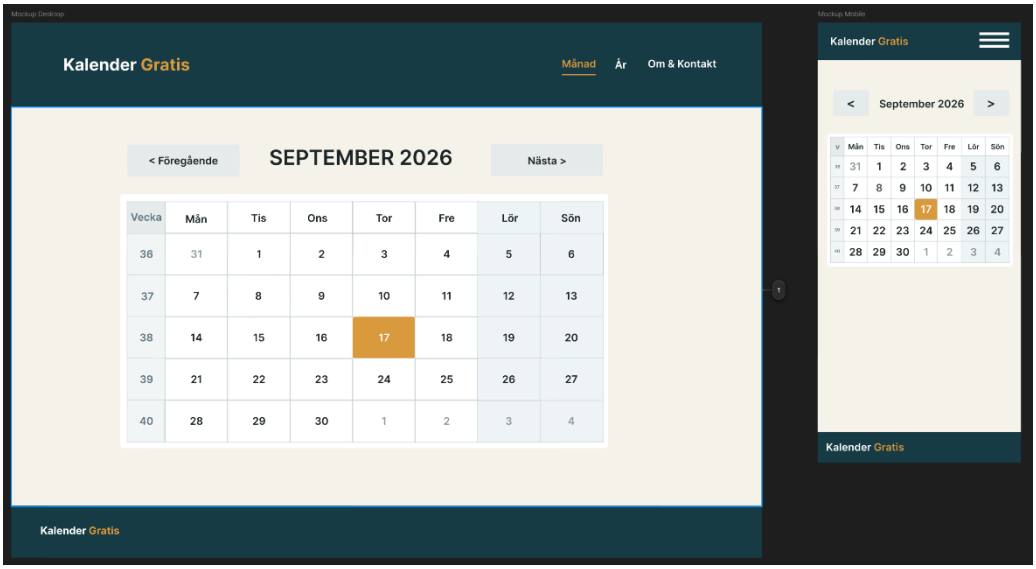
4 Metod

4.1 Förstudie och planering

Arbetet inleddes med en jämförelse av befintliga svenska kalendersidor och en avgränsning av vilka funktioner som skulle prioriteras. Därefter ritades wireframes för månad, år och kontakt i både mobil- och desktopformat. En färgsatt mockup togs sedan fram i Figma.



Figur 1. Wireframes för tre sidor i desktop och mobilformat.



Figur 2. Mockup på design och färger.

4.2 Iterativ implementation

Implementationen och testningen genomfördes i små steg. Först skapades en grundstruktur i HTML, navigation och footer. Sedan bygges kalenderrutnätet upp statisk för att i nästa steg ersättas med dynamisk generering med hjälp av JavaScript. Årsvy, länkar mellan vyerna, kontaktformulär, API-data med helg och namnsdagar, felhantering och cache lades till successivt. Efter varje ny fungerande funktion så kontrollerades resultatet. Efter varje implementation av en

ny funktion eller ändring av sidan så kontrollerades resultatet i webbläsare och commitades till Github.

4.3 Verktyg och testmetoder

- Design i form av wireframes och mockup för de grafiska delarna gjordes i Figma.
- Kod skrevs i Visual Studio Code och för felsökning användes webbläsarnas (Chrome/Firefox) utvecklarverktyg.
- Versionshistorik hanteras via Git mot Github i main/dev grenar.
- Validering och kontroll av kod skedde med W3C HTML[6] och CSS[7] validatorer.
- Tillgänglighet och responsivitet kollades med WAVE[8], datorns tangentbord och olika vybredder.

[6] W3C. Nu Html Checker. Tillgänglig: https://validator.w3.org/nu/ [Hämtad 2026-10-30].
[7] W3C. The W3C CSS Validation Service. Tillgänglig: https://jigsaw.w3.org/css-validator/ [Hämtad 2026-10-30].
[8] WebAIM. WAVE Web Accessibility Evaluation Tools. Tillgänglig: https://wave.webaim.org/ [Hämtad 2026-10-30].

5 Konstruktion

5.1 Informationsarkitektur

Webbplatsen delades in i tre sidor. Månadsvyn fungerar som startsida och visar det som bedömdes som mest relevant att se från en kalender. Årsvyn delar hela året i tolv stycken mindre kalendrar för att ge en bättre översikt. Kontakt / Om sidan visar information om tjänsten och kontakt information. Samma header, navigation, footer och grafisk profil används på samtliga sidor.

5.2 Grafisk profil och responsiv layout

Den grafiska profilen har en mörk petrolfärg för header och footer, en ljus bakgrund och visa innehållsrutor och en ockragul accent. Färgvariablerna i root gör paletten konsekvent och lätt att justera. Den dämpade textfärgen ändrades från #8a999d till #596a6e efter ett kontrasttest med WAVE. Månadskalendern har åtta kolumner, en smalare för veckonummer och sju lika breda för dagar. Den visar konstant sex stycken veckor för att hålla samma form oavsett vilken månad som visas. Årsvyn visar en, två eller tre månader i bredd baserat på skärmupplösning. Logotypen skapades med hjälp av ChatGPT bildgenereringsverktyg utifrån webbplatsens färgpalett och grafiska profil. Det genererade förslaget valdes och anpassades för användning i webbplatsens sidhuvud.

5.3 Beräkning av kalendern

Månadens första dag skapas med Date. Eftersom JavaScript räknar söndag som dag noll så räknas ett startOffset om så att måndag blir första kolumn. Klandern backar till den måndagen och skapar därefter alltid sex veckor. Den fasta höjden gör att layouten inte hoppar när användaren byter månad. Datumen från föregående och kommande månad visas dämpat utan helgdagar och namnsdagar. Samma princip återanvänds senare för varje kalender i årsvyn.

```
const startOffset = (firstDayOfMonth.getDay() + 6) % 7;
const calendarDate = new Date(displayedYear, displayedMonth, 1 -
startOffset);
const numberOfWeeks = 6;
```

Kodexempel 1. Beräkning av första datum och fast antal kalenderveckor

5.4 Extern kalenderdata och cache

Svenska dagar API används för att hämta namnsdagar, helgdagar, röda dagar och arbetsfria dagar.[9] Varje datumobjekt matchas mot nyckeln ÅÅÅÅ-MM-DD. Röda dagar får en egen CSS klass. Midsommarafton, julafton och nyårsafton identifieras som arbetsfria dagar som inte är röda och får en separat markering. Map används som cache med årtal som nyckel. Om användaren bläddrar mellan månader inom samma år så görs ett anrop vid första månaden och sedan hämtas information för andra månader från cache. När användaren går över till ett annat år så görs ett nytt API anrop. Detta för att minska belastning på API och göra navigationen mellan månader snabbare.

```
const calendarDataCache = new Map();
```

[9] Svenska Dagar API. Dokumentation och API för svenska kalenderdagar. Tillgänglig: https://sholiday.faboul.se/ [Hämtad 2026-10-30].

```
async function loadCalendarData(year) {
  if (calendarDataCache.has(year)) {
    return calendarDataCache.get(year);
  }
  // fetch, kontroll av response.ok och JSON-tolkning
}
```

Kodexempel 2. Principen för årsbaserad cache av API-data.

5.5 Status och felhantering

När data för helgdagar och namnsdagar hämtas så visas ”Hämtar klanderinformation” i en statusrad ovanför kalenders rutnät. Vid fel så visar den statusraden ett felmeddelande samt att sidan genereras med dagar men utan information om namnsdagar och helgdagar. Det innebär att kärnfunktionen fungerar även om API-anropet misslyckas.

5.6 Navigation och formulär

Månadsvyn och kalendervyn länkas samma med URL-parametrarna year och month. En användaren kan exempelvis öppna Oktober 2026 ifrån åsvyn och behålla rätt år även efter navigering. Mobilmenyn växklar CSS-klassen open och uppdaterar ariaexpanded. Kontakforumläret kontrollerar namn, e-postadress och meddelandeläng. Vid fel så visas ett beskrivande felmeddelande vid det aktuella fältet och formuläret godkänds endast när samtliga fält är giltiga.

5.7 Tillgänglighetsval

- Svenska anges med lang=”sv”
- Varje sida har en tydlig h1 rubrik
- Navigationsknapparna har begripliga aria-label attribut och dekorativa symboler är aria-hidden
- Dynamiska statusmeddelanden använder role=”status” och aria-live=”polite”
- Kalenderdagar får läsbara beskrivningar med datum, helgdag och namnsdag
- Fokuslägen, textstorlek och kontrast har kontrollerats och justerats.

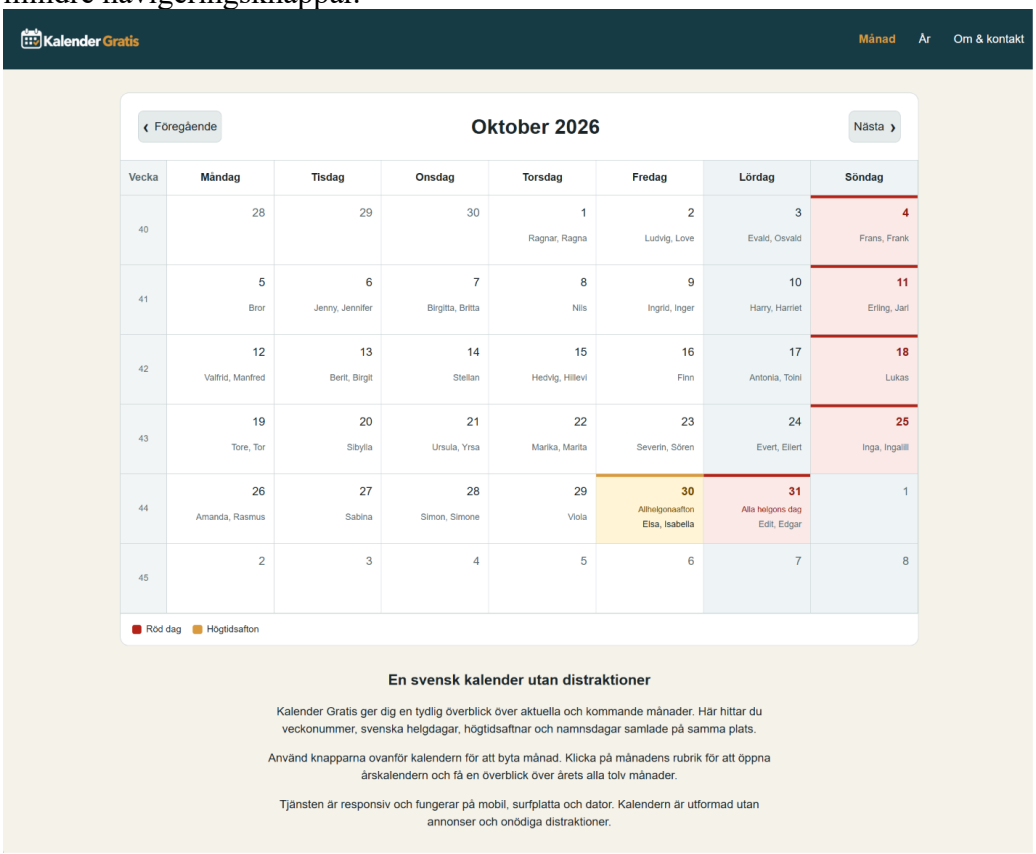
6 Resultat

6.1 Färdig funktionalitet

Resultatet är en responsiv kalenderwebbplats bestående av tre sidor. Kalendern visar datum, veckonummer, helgdagar, högtidsaftnar och namnsdagar. Användaren kan navigera mellan månader och år, och innehållet anpassas efter skärmens storlek. Webbplatsen innehåller även en mobilmeny, ett validerat kontaktformulär och felhantering för den externa kalenderinformationen.

6.1.1 Månadsvy

Månadsvyn visar aktuell månad med veckonummer, helgdagar och namnsdagar. Knapparna Föregående och Nästa används för att byta månad, medan månadens rubrik länkar till årsvyn. Mobilversionen använder kortare veckodagsnamn och mindre navigeringsknappar.



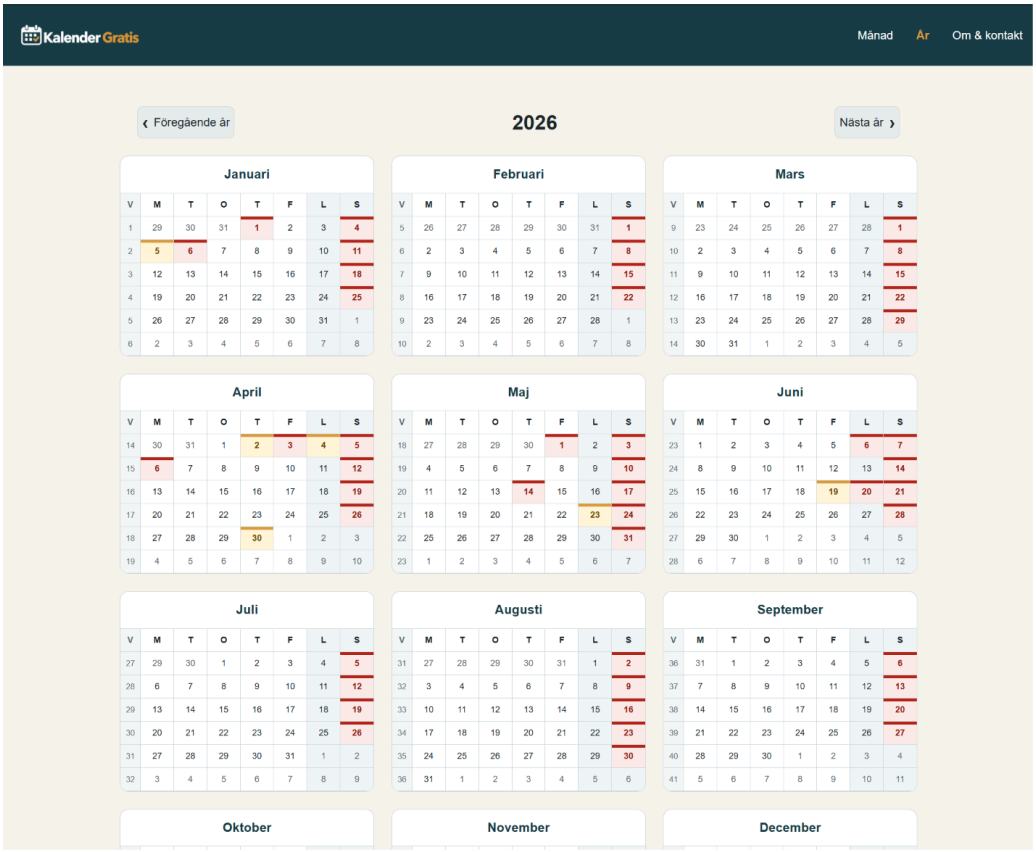
Figur 3. Månadsvy i desktopläge.



Figur 4. Månadvy i mobilläge.

6.1.2 Årsvy

Årsvyn visar årets tolv månader och låter användaren navigera mellan olika år. Månadsnamnen fungerar som länkar till motsvarande månad i månadsvyn. Layouten visar en, två eller tre kolumner beroende på skärmens bredd.



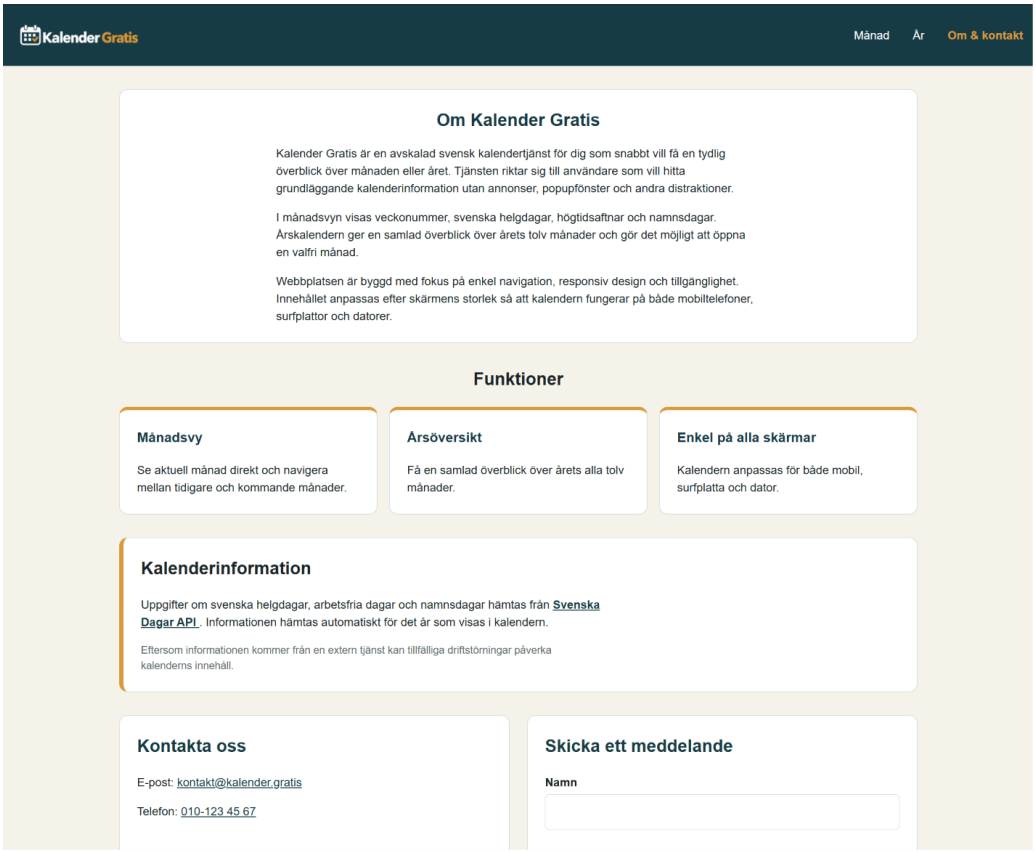
Figur 5. Kalendervy i desktopläge.



Figur 6. Årsvy i mobilläge.

6.1.3 Om och Kontakt

Sidan presenterar webbplatsens syfte, funktioner, datakälla och kontaktuppgifter. Kontaktformuläret valideras med JavaScript och visar felmeddelanden vid ogiltig inmatning. På mobil placeras innehållet i en kolumn och navigationen öppnas med menyknappen.



Figur 7. Om och kontakt sida i desktopläge.



Figur 8. Om och Kontaktsida i mobilläge med öppen navigationsmeny.

6.2 Testresultat

Test	Resultat	Kommentarer
HTML-validering*	Godkänt	Inga fel
CSS-validering	Godkänt	Inga fel och nivå 3 + SVG
WAVE, samtliga sidor	10/10	Noll error och noll contras errors
Tangentbordsnavigation	Godkänt	Meny, länkar, knappar och formulär kan nås
Responsiv layout	Godkänt	Kontrollerad i olika visningsbredder

Års och månadsskifte	Godkänt	Behåller rätt datum vid navigering mellan vyer, månader och år
API-fel	Godkänt	Meddelar om fel och ger grundfunktion av kalendern vid API-fel

*Validering gav info punkter gällande formatering orsakad av Prettier i VS Code, dessa lämnades utan åtgärd.

6.3 Versionshistorik

Github-repot innehåller 25 commits under perioden 23-28e oktober 2026. Historiken visar hur sidan successivt har utvecklats och när funktioner har tillkommit.

Källkoden finns på: https://github.com/Svenlaban/Kalender-gratis/tree/main

Och den publicerade webbsidan: https://kalender.gratis

7 Slutsatser

Projektets mål att skapa en enkel och responsiv svensk kalendertjänst har i huvudsak uppnåtts. Den färdiga webbplatsen erbjuder både månads- och årsvy samt information, kontaktuppgifter och ett validerat formulär. Testresultaten visar att webbplatsen fungerar på olika skärmstorlekar och att HTML, CSS och grundläggande tillgänglighet uppfyller sidans syfte och uppgiftens mål.

En större förändring under projektet hände när API:et för namnsdagar skulle implementeras och det upptäcktes att det att Svenska dagar API även innehöll information om röda dagar, tidigare användes ett annat API för att hämta röda dagar. Då fick den funktionaliteten byggas om för att enbart använda ett API för att få in informationen och använda den nya datastrukturen. Detta medförde att det blev en del extraarbete och felsökning innan både namnsdagar och helgdagar fungerade från den nya API tjänsten.

Tillgänglighetstesterna visade att en design kan se tydlig ut utan att färgkontrasten är tillräcklig. Efter resultaten från WAVE ändrades bland annat den dämpade textfärgen och namnsdagarnas textstorlek. Det blev därför tydligt att validering och tillgänglighet behöver kontrolleras under hela utvecklingen och inte enbart när webbplatsen är färdig.

Alla delar fungerar inte helt som avsett. En kvarvarande brist är att markeringen av dagens datum inte syns korrekt när datumet samtidigt är en röd dag eller högtidsafton. Markeringarnas stilar påverkar då varandra, vilket behöver rättas i en kommande uppdatering. En funktion där användaren kunde klicka på ett datum för att visa mer information fanns det en tanke om, men valdes bort eftersom nyttan bedömdes vara begränsad, framför allt på desktop. Även detta är en funktion som kommer tittas på i kommande uppdateringar.

Webbplatsen saknar lagring av egna händelser och kontaktformuläret skickar inga uppgifter till en server. Detta är en funktion som implementerades för att efterleva uppgiftens grundkrav och kommer sannolikt tas bort i kommande uppdateringar.

Källförteckning

[1] WHATWG. HTML Living Standard: Semantics, structure, and APIs of HTML documents. Tillgänglig: https://html.spec.whatwg.org/multipage/semantics.html [Hämtad 2026-10-29].

[2] World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.2. Tillgänglig: https://www.w3.org/TR/WCAG22/ [Hämtad 2026-10-29].

[3] MDN Web Docs. CSS media queries. Tillgänglig: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries [Hämtad 2026-10-29].

[4] MDN Web Docs. Fetch API. Tillgänglig: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API [Hämtad 2026-10-29].

[5] Chacon S, Straub B. Pro Git. 2:a uppl. Tillgänglig: https://git-scm.com/book/en/v2 [Hämtad 2026-10-29].

[6] W3C. Nu HTML Checker. Tillgänglig: https://validator.w3.org/nu/ [Hämtad 2026-10-30].

[7] W3C. The W3C CSS Validation Service. Tillgänglig: https://jigsaw.w3.org/css-validator/ [Hämtad 2026-10-30].

[8] WebAIM. WAVE Web Accessibility Evaluation Tools. Tillgänglig: https://wave.webaim.org/ [Hämtad 2026-10-30].

[9] Svenska Dagar API. Dokumentation och API för svenska kalenderdagar. Tillgänglig: https://sholiday.faboul.se/ [Hämtad 2026-10-30].